0056. Merge Intervals

Medium | Category: Intervals | LeetCode: https://leetcode.com/problems/merge-

Problem

- Merge all overlapping intervals.

Intuition

- After sorting by start, overlapping intervals are adjacent. Extend the last merged interval while the next one starts before it ends.

Brute Force

- Compare every pair repeatedly until nothing merges: O(n^2) or worse.

Optimal Approach

- Sort by start.
- For each interval: if it starts <= last.end, last.end = max(last.end, end); else append it.

Worked Example

```
[[1,3],[2,6],[8,10],[15,18]].

[1,3]   -> res [[1,3]]
[2,6]   2 <= 3 -> [1,6]
[8,10]  new
[15,18] new
result [[1,6],[8,10],[15,18]]
```

Java Solution

```java
public class MergeIntervals {
    public int[][] merge(int[][] intervals) {
        int[][] a = intervals.clone();
        Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));
        List<int[]> res = new ArrayList<>();
        for (int[] cur : a) {
            if (!res.isEmpty() && cur[0] <= res.get(res.size() - 1)[1]) {
                int[] last = res.get(res.size() - 1);
                last[1] = Math.max(last[1], cur[1]);
            } else {
                res.add(new int[]{cur[0], cur[1]});
            }
        }
        return res.toArray(new int[0][]);
    }
}
```

Complexity

- Time: O(n log n). Space: O(n).

Pitfalls

- Use max for the end: [1,10] then [2,3] must stay [1,10].